

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЕТ
к лабораторной работе № 1
на тему
ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ ПО
Вариант 2

Студент:

М.А. Тукайло

Преподаватель:

Д.В. Куприянова

Минск 2026

1 ЦЕЛЬ И ЗАДАЧИ

Целью работы является изучение методов работы в среде разработки IntelliJ IDEA при создании приложения на объектно-ориентированном языке программирования Java, разработка приложения с графическим интерфейсом, изучение архитектурного паттерн MVC с активной и пассивной моделью.

В соответствии с вариантом 2 необходимо разработать астрологическое приложение для расчета судьбы человека по дню недели его рождения, с вводом даты через собственную форму, вычислением дня недели и отображением результата на главной форме приложения. Архитектура реализуется по паттерну MVC с активной моделью. Для выполнения работы поставлены следующие задачи:

1 Создать и настроить проект Java в IntelliJ IDEA, изучить средства редактирования, сборки, запуска и отладки.

2 Подобрать средства IntelliJ IDEA для работы с UML и SysMLv2 и отразить результат настройки в отчете.

3 Разработать графическое приложение с главной формой и отдельным окном для ввода даты рождения, не используя стандартный элемент управления для ввода даты.

4 Обеспечить запоминание последних введенных данных на главной форме и их восстановление при повторном вводе.

5 Реализовать проверку корректности введенных данных и вывод понятного сообщения об ошибке при некорректном вводе.

6 Разделить приложение на модель, представление и контроллер и реализовать активное уведомление представления моделью.

7 Проверить основные сценарии работы программы и подготовить ответы на контрольные вопросы.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЕ

2.1 Интегрированная среда разработки IntelliJ IDEA

IntelliJ IDEA – это интегрированная среда разработки (IDE), созданная чешской компанией JetBrains, предназначенная в первую очередь для разработки на языке Java, а также поддерживающая Kotlin, Scala, Groovy и множество других языков программирования через систему плагинов.

Основная ценность IntelliJ IDEA заключается в глубоком статическом анализе кода: среда постоянно анализирует исходный код проекта в фоновом режиме, что позволяет ей предлагать умное автодополнение (учитывающее контекст использования), мгновенно находить ошибки и потенциальные проблемы еще до компиляции, а также предоставлять мощные инструменты рефакторинга.

Компилятор `javac` преобразует исходный код Java в байт-код, исполняемый виртуальной машиной JVM. В IntelliJ IDEA сборка проекта выполняется командой `Build Project`, а запуск и отладка класса `App` осуществляются через команды `Run` и `Debug` соответственно. Для остановки выполнения программы в нужном месте устанавливается точка останова – щелчком в поле слева от строки кода либо сочетанием клавиш `Ctrl+F8`. Когда выполнение приостановлено, окно `Debug` отображает стек вызовов и текущие значения переменных, а пошаговое перемещение по коду обеспечивается командами `Step Over`, `Step Into` и `Step Out`.

Для работы с UML и SysMLv2 недостающие инструменты можно установить через `Plugins`.

2.2 Паттерн MVC

MVC (Model-View-Controller) – архитектурный паттерн, разделяющий приложение на три компонента. `Model` хранит данные и реализует предметную логику – проверку корректности данных и вычисления. `View` формирует интерфейс и отображает данные, не выполняя вычислений. `Controller` обрабатывает действия пользователя и координирует взаимодействие между `Model` и `View`.

2.3 Активная и пассивная модель

Архитектурное решение MVC – способ взаимодействия модели и представления. При пассивной модели контроллер сам инициирует обновление представления после изменения данных. Активная модель, напротив, самостоятельно уведомляет об изменении своего состояния, освобождая контроллер от этой задачи. Такой подход снижает связанность между компонентами приложения и делает представление более независимым от логики контроллера.

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Выбор языка программирования и среды разработки

В качестве объектно-ориентированного языка программирования для выполнения работы выбран Java – язык со строгой типизацией, полностью поддерживающий принципы ООП и обладающий развитой экосистемой, удобной для изучения архитектурных паттернов, включая MVC.

В качестве среды разработки выбрана IntelliJ IDEA (JetBrains), предоставляющая удобные инструменты автодополнения, рефакторинга, анализа кода и мощный визуальный отладчик, а также бесплатную версию Community Edition, достаточную для выполнения задания.

3.2 Установка и настройка среды разработки

Установка IntelliJ IDEA производилась путем загрузки дистрибутива с официального сайта JetBrains (jetbrains.com/idea), представленного на рисунке 3.2.1. Для операционной системы Windows был выбран исполняемый файл формата .exe.

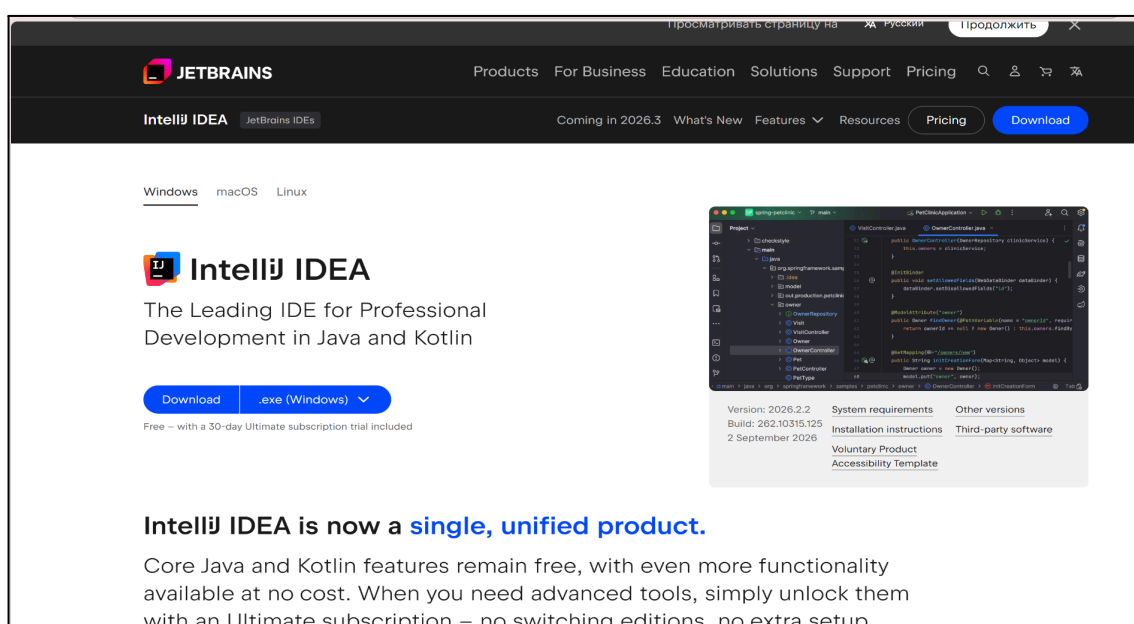


Рисунок 3.2.1 – Страница загрузки IntelliJ IDEA

После запуска установочного файла был запущен мастер установки IntelliJ IDEA Setup, представленный на рисунке 3.2.2. На этапе настройки параметров установки (Installation Options) были выбраны следующие опции: создание ярлыка на рабочем столе, добавление пункта «Open Folder as Project» в контекстное меню, а также создание ассоциаций с файлами типов .java и .gradle.

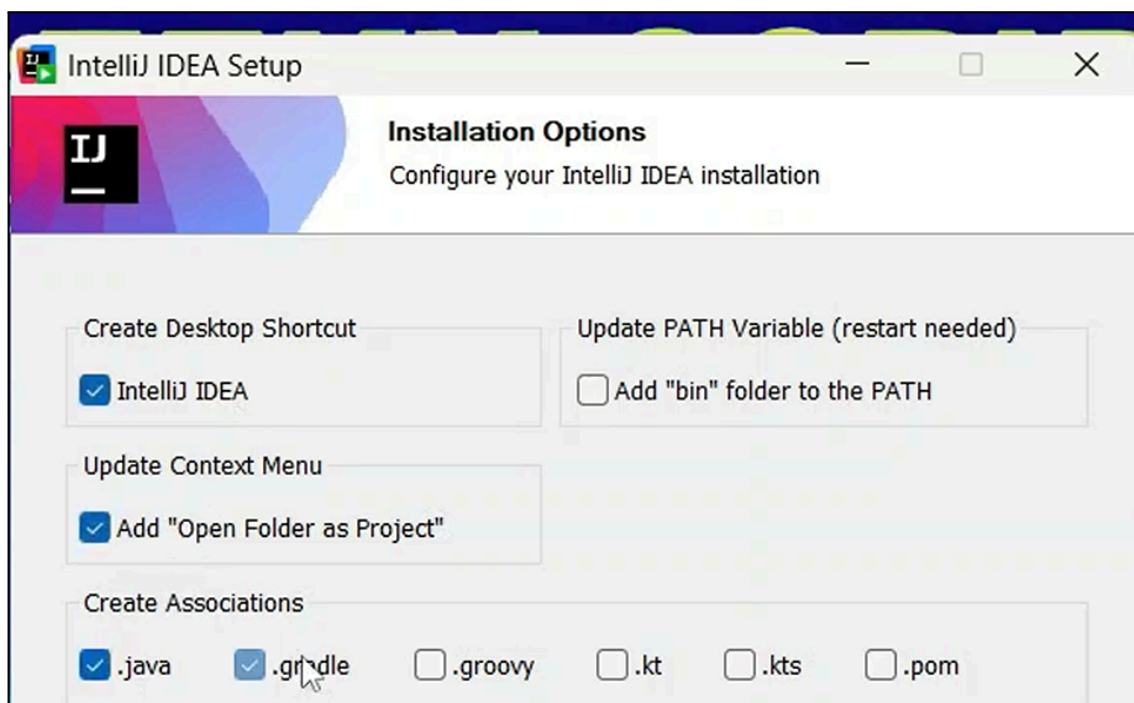


Рисунок 3.2.1 – Окно настройки параметров установки IntelliJ IDEA

После завершения работы мастера установки IDE была запущена, выполнена первоначальная настройка (выбор темы оформления, установка JDK).

3.2 Создание проекта

После создания нового Java-проекта была выполнена его дополнительная настройка: указан язык проекта – Java 17, определена структура каталогов согласно стандартной схеме IntelliJ IDEA (директория src для исходного кода, out/out/production – для скомпилированных классов). В настройках проекта были проверены параметры SDK, уровень языка (Language level) и кодировка исходных файлов (UTF-8). После этого в проекте был создан базовый пакет и стартовый класс с методом main, что позволило убедиться в корректности настройки среды перед выполнением дальнейших пунктов задания.

3.3 Настройка среды и расширений для UML и SysML v2

В IntelliJ IDEA открыт проект fate с каталогом src/main и пакетами main.controller, main.model, main.view. В качестве Project SDK выбран JDK 17. Класс main.Main содержит метод main и используется в конфигурации запуска. Для UML применяется плагин PlantUML, представленный на рисунке 3.3.1, и файл docs/diagram.puml. Для материалов SysML v2 в IntelliJ IDEA специализированного плагина не найдено, поэтому используется текстовая нотация языка.

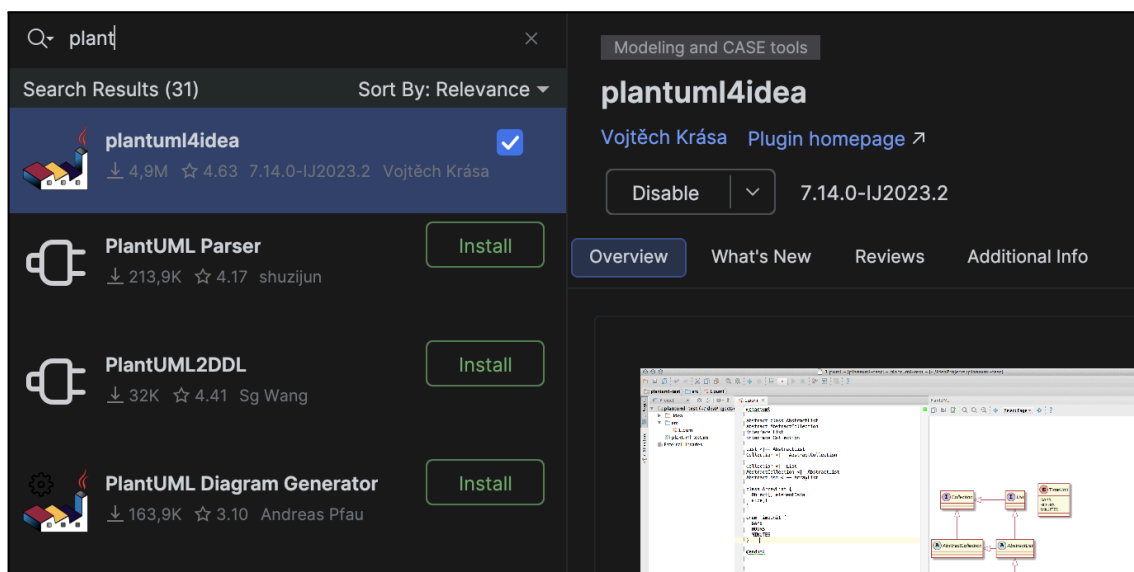


Рисунок 3.3.1 – Средства IntelliJ IDEA для работы с UML и SysMLv2

Структурное представление системы в нотации, близкой к SysMLv2:

```
package Fate {
    part def MainController;
    part def DateBirth;
    part def Fate;
    part def FateRepository;
    part def MainForm;
    part def DateInputDialog;

    part application {
        part controller : MainController;
        part model : DateBirth;
        part fate : Fate;
        part repository : FateRepository;
        part view : MainForm;
        part dialog : DateInputDialog;

        connect view to controller;
        connect controller to model;
        connect controller to dialog;
        connect model to view;
        connect repository to fate;
        connect view to repository;
    }
}
```

3.4 Демонстрация отладчика

Точка останова установлена и продемонстрирована на рисунке 3.4.1 в методе `onInputDate` класса `MainController`. В окне `Debug` видны значения переменных `day`, `month`, `year`, введенных пользователем. Пошаговое

выполнение позволяет проследить проверку даты и обновление модели, а функция Evaluate Expression – проверить результат метода `getDayOfWeek()`.

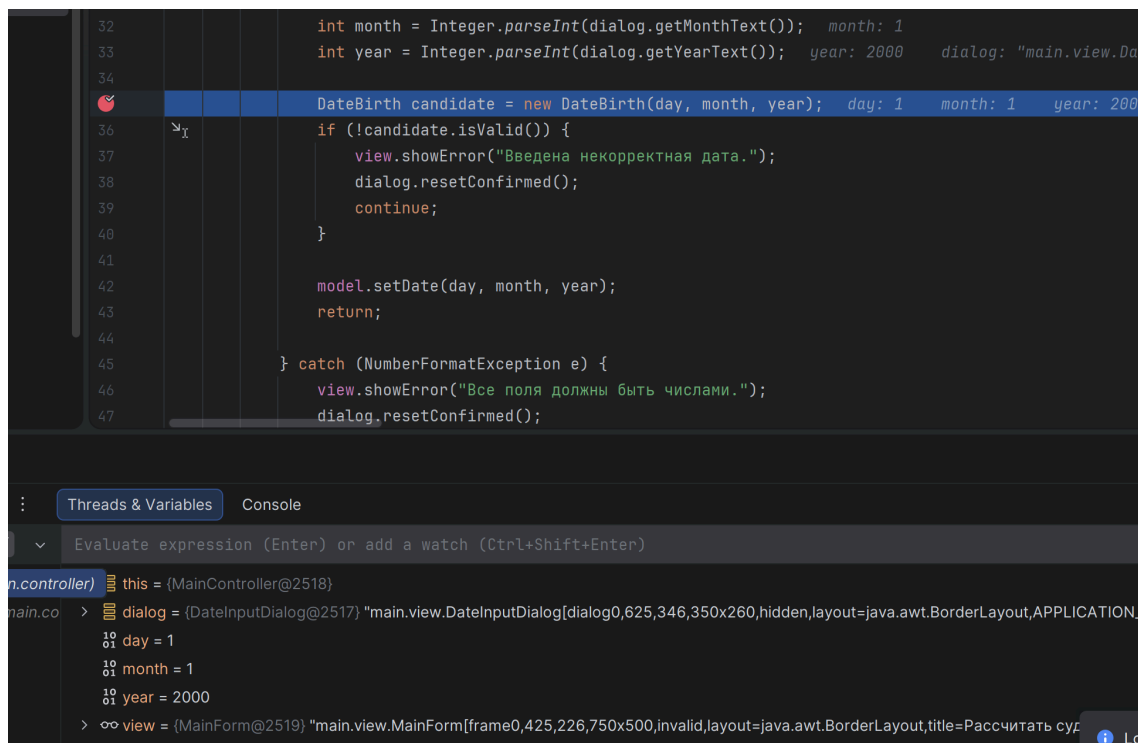


Рисунок 3.4.1 – Остановка программы в отладчике IntelliJ IDEA

3.5 Структура проекта и каркас MVC с активной моделью

Разработанное приложение построено на основе архитектурного паттерна MVC (Model-View-Controller), что обеспечивает разделение ответственности между компонентами системы и упрощает дальнейшее сопровождение и расширение функциональности. Каждый компонент отвечает за свою отдельную задачу, а взаимодействие между ними реализовано через четко определенные интерфейсы и механизмы уведомлений, представленные на рисунке 3.5.1.

Проект разделен на следующие компоненты:

1 DateBirth – хранит дату рождения и вычисленный результат, содержит бизнес-логику проверки даты и вычисления дня недели, механизм уведомления слушателей об изменении данных.

2 MainForm – главный интерфейс, подписывается на изменения модели для автоматического обновления результата на экране.

3 DateInputDialog – модальное окно для ввода даты рождения.

4 MainController – обрабатывает ввод пользователя, выполняет парсинг строк в числа и валидацию, передает данные в модель.

5 Fate / FateRepository – хранят и предоставляют текстовые описания судьбы для каждого дня недели.

6 Main – точка входа, связывающая компоненты между собой.

Интерфейсы и механизмы уведомлений, представленные на рисунке 3.5.1.

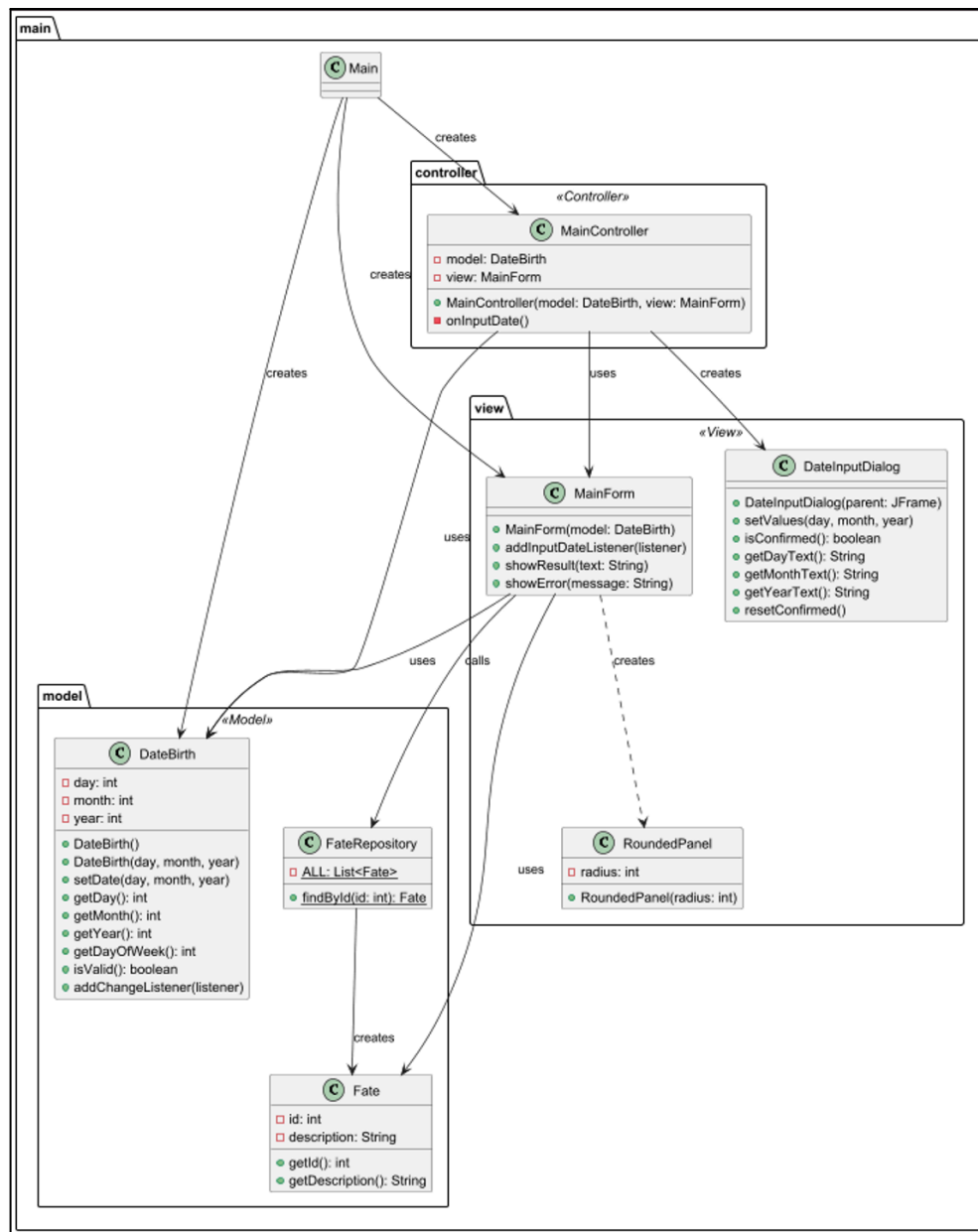


Рисунок 3.5.1 – Диаграмма типов и интерфейсов приложения

3.6 Модель

`DateBirth` реализует активную модель. Метод `isValid()` проверяет фактическое существование введенной даты через `LocalDate.of()`, а также то, что дата не превышает текущий момент (`!date.isAfter(LocalDate.now())`). После успешной валидации в контроллере вызывается `setDate()`, которая обновляет значения `day`, `month`, `year` и уведомляет подписчиков об изменении через `PropertyChangeSupport`. Метод `getDayOfWeek()` вычисляет день недели рождения, который в дальнейшем используется для определения соответствующей судьбы.

3.7 Представление

MainForm отображает результат – текстовое описание судьбы, соответствующее дню недели введенной даты. Модель DateBirth уведомляет MainForm об изменении данных через addChangeListener, после чего в обработчике происходит обращение к FateRepository.findById() и обновление текста через метод showResult(). DateInputDialog обеспечивает ввод даты рождения в отдельном модальном окне с тремя полями (день, месяц, год) и кнопкой подтверждения.

3.8 Контроллер

MainController управляет жизненным циклом DateInputDialog и отвечает за безопасный перевод пользовательского ввода из текстового формата в числовой. В случае NumberFormatException контроллер перехватывает ошибку и выводит сообщение пользователю через view.showError(), после чего диалог остается открытым для повторного ввода (dialog.resetConfirmed()).

3.9 Запоминание данных и обработка ошибок

Приложение предотвращает ввод некорректных дат: если введенная дата не существует или относится к будущему, метод isValid() возвращает false, а пользователю выводится сообщение «Введена некорректная дата.». Аналогично обрабатывается ситуация, когда поля содержат нечисловые значения – выводится сообщение «Все поля должны быть числами.». Восстановление ранее введенных значений реализовано через метод dialog.setValues(), который вызывается перед повторным показом окна ввода, подставляя туда текущие данные модели.

4 РЕЗУЛЬТАТЫ РАБОТЫ

В IntelliJ IDEA разработано и протестировано Java-приложение. Программа определяет по введенной дате рождения день недели и выводит соответствующее текстовое описание судьбы, сохраняя последние корректно введенные данные и сообщая пользователю о некорректном вводе. На рисунке 4.1 представлена главная форма после запуска приложения и на рисунке 4.2 показано окно ввода данных.

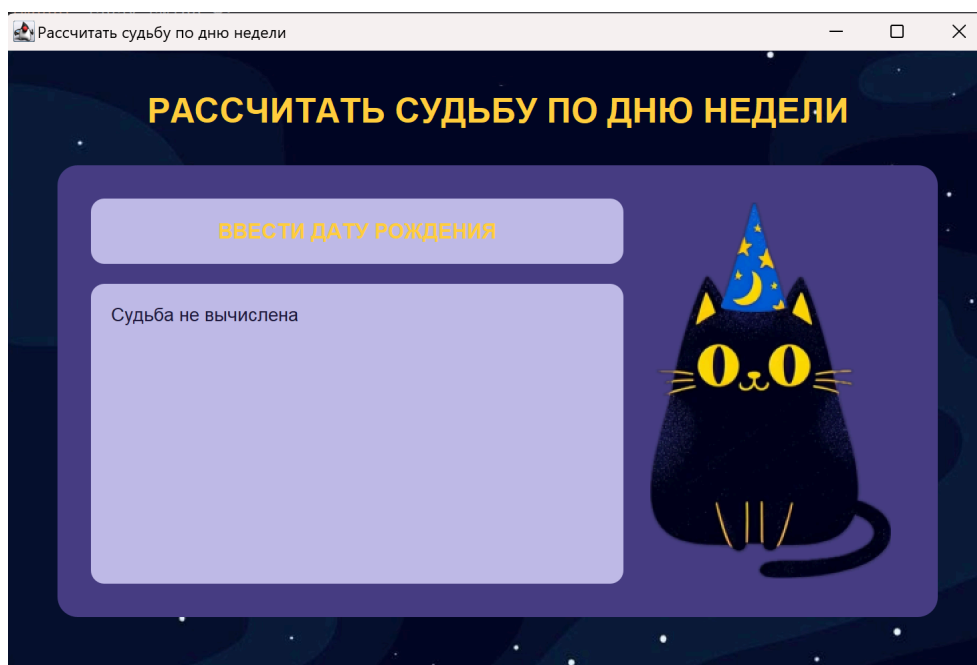


Рисунок 4.1 – Главная форма после запуска

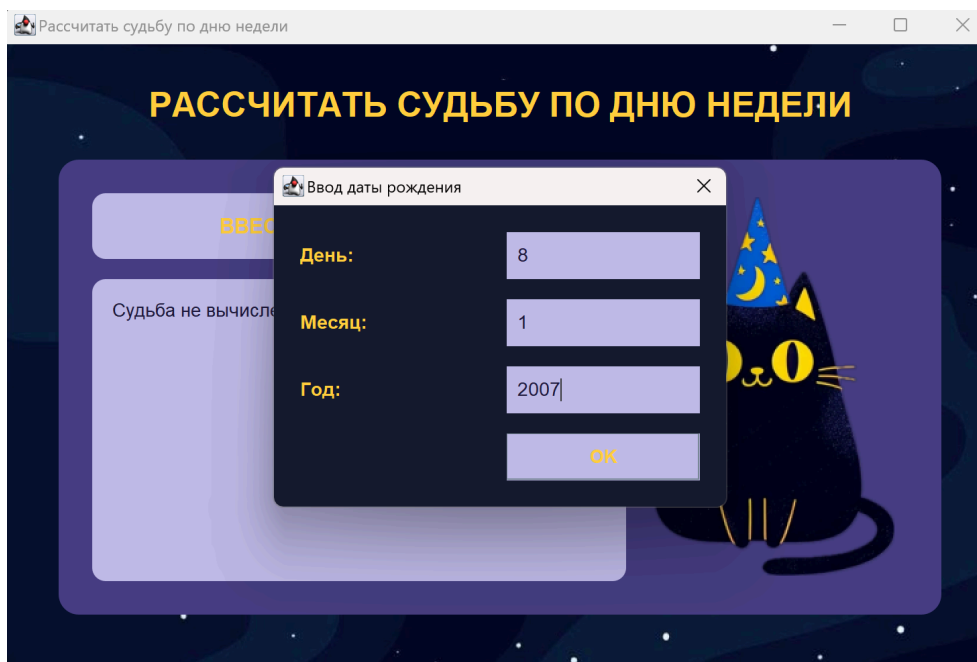


Рисунок 4.2 – Окно ввода данных

На рисунке 4.3 представлен результат вычисления судьбы.

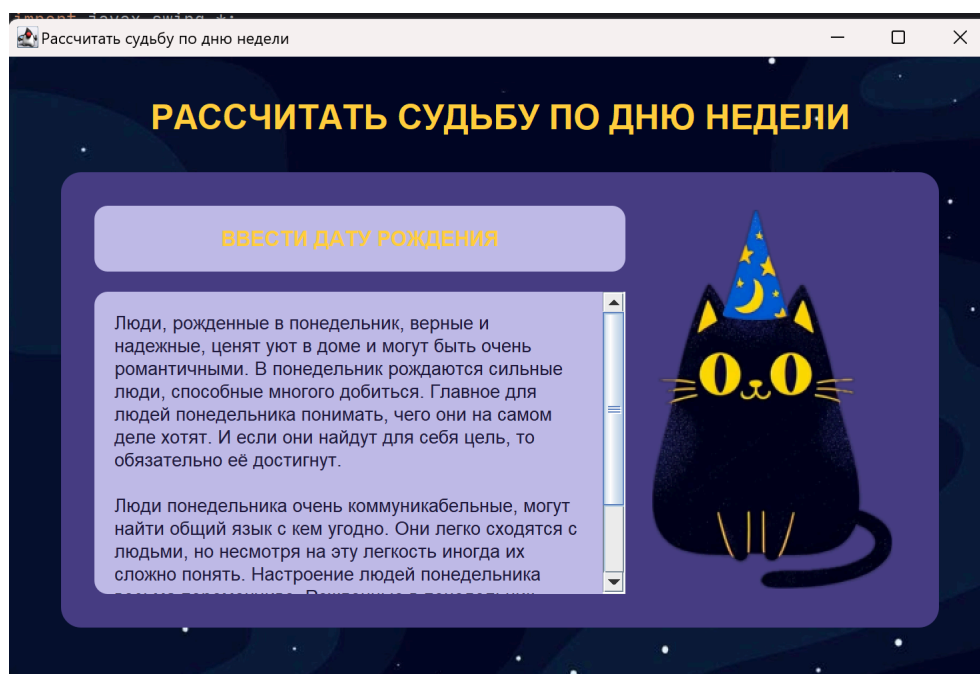


Рисунок 4.3 – Результат вычисления

На рисунке 4.4 представлена демонстрация ошибки.

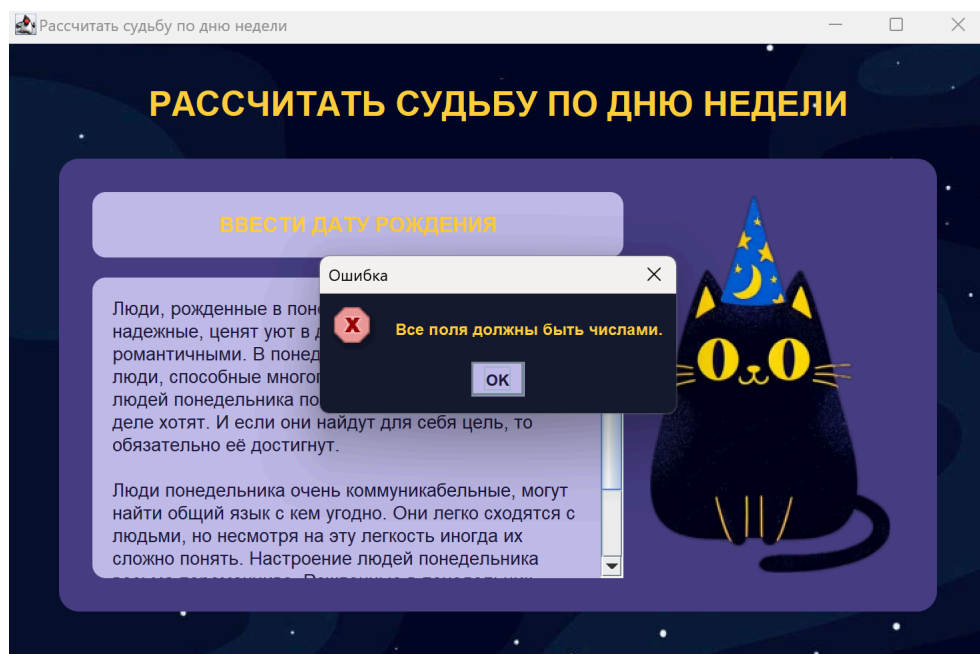


Рисунок 4.4 – Демонстрация ошибки

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

5.1 Какие компоненты входят в состав IDE?

Интегрированная среда разработки (IDE) обычно включает: редактор кода, компилятор/интерпретатор или интеграцию с ним, отладчик (debugger), систему сборки, средства для работы с системами контроля версий (Git), инструменты рефакторинга, встроенную документацию/справку, а также средства для запуска и тестирования приложения.

5.2 Каковы основные приемы работы с редактором кода?

К основным приемам относятся: автодополнение кода, подсветка синтаксиса и ошибок, быстрая навигация по коду, рефакторинг, форматирование кода по стандартам, использование горячих клавиш для ускорения работы, а также встроенные подсказки и статический анализ кода в реальном времени.

5.3 Каким образом выполняется сборка и запуск проекта?

Сборка проекта выполняется либо встроенными средствами IDE (компиляция .java-файлов в байт-код), либо через систему сборки (Maven/Gradle), которая управляет зависимостями и этапами сборки. Запуск осуществляется через настроенную конфигурацию запуска (Run Configuration), которая указывает главный класс с методом main, параметры JVM и аргументы командной строки. После нажатия кнопки Run IDE компилирует измененный код и запускает приложение в отдельном процессе.

5.4 Что такое MVC?

MVC (Model-View-Controller) – архитектурный паттерн, разделяющий приложение на три компонента: Model (модель) – хранит данные и бизнес-логику приложения. View (представление) – отвечает за отображение данных пользователю. Controller (контроллер) – обрабатывает действия пользователя, обновляет модель и/или представление. Такое разделение обеспечивает независимость компонентов друг от друга, упрощает тестирование и сопровождение кода.

5.5 Чем отличается активная модель от пассивной?

Пассивная модель хранит данные и предоставляет методы доступа к ним, но не уведомляет об изменениях – компоненты должны сами запрашивать ее состояние. Активная модель самостоятельно оповещает об изменениях через механизм слушателей, поэтому представление обновляется автоматически, без явного опроса.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была освоена среда разработки IntelliJ IDEA: выполнена ее установка и первоначальная настройка, изучены основные функциональные возможности редактора кода и средства отладки на примере разработанного Java-приложения.

Создан проект fate, реализующий архитектуру MVC с активной моделью: контроллер MainController обрабатывает пользовательский ввод, модель DateBirth выполняет валидацию и вычисление дня недели рождения, а представление MainForm автоматически обновляется при изменении модели через механизм PropertyChangeSupport.

Продемонстрирована работа отладчика – установка точек останова, пошаговое выполнение кода и использование функции Evaluate Expression для проверки промежуточных вычислений.

Для визуализации структуры проекта построена UML-диаграмма классов с использованием плагина PlantUML.

Таким образом, все задачи, поставленные в рамках лабораторной работы, были выполнены: изучена среда разработки, разработано рабочее приложение с архитектурой MVC, построена UML-диаграмма и проанализированы возможности инструментов моделирования, доступных в IntelliJ IDEA.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг исходного кода приложения

Main.java

```
package main;

import main.controller.MainController;
import main.model.DateBirth;
import main.view.MainForm;

import javax.swing.*.*;
import java.awt.*.*;

public class Main {
    public static void main(String[] args) {

        DateBirth model = new DateBirth();
        MainForm view = new MainForm(model);
        MainController controller = new MainController(model,
view);
        view.setVisible(true);
    }
}
```

RoundedPanel.java

```
package main.view;

import javax.swing.*.*;
import java.awt.*.*;

public class RoundedPanel extends JPanel {

    private final int radius;

    public RoundedPanel(int radius) {
        this.radius = radius;
        setOpaque(false);
    }

    @Override
    protected void paintComponent(Graphics g) {
        Graphics2D g2 = (Graphics2D) g.create();
```

```

        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
        g2.setColor(getBackground());
        g2.fillRoundRect(0, 0, getWidth(), getHeight(), radius,
radius);
        g2.dispose();
        super.paintComponent(g);
    }
}

```

MainForm.java

```

package main.view;

import main.model.DateBirth;
import main.model.Fate;
import main.model.FateRepository;

import javax.swing.*;
import java.awt.*;
import java.net.URL;

public class MainForm extends JFrame {

    private static final Color PANEL_PURPLE = new Color(70, 62,
130);
    private static final Color FIELD_LIGHT = new Color(190, 185,
230);
    private static final Color ACCENT_YELLOW = new Color(255,
205, 60);
    private static final Color TEXT_DARK = new Color(30, 25,
60);

    private JButton inputDateButton;
    private JTextArea resultArea;
    private Image backgroundImage;

    public MainForm(DateBirth model) {

        UIManager.put("OptionPane.background", new Color(20, 25,
45));
        UIManager.put("Panel.background", new Color(20, 25,
45));
    }
}

```

```

        UIManager.put("OptionPane.messageForeground", new
Color(255, 205, 60));
        UIManager.put("Button.background", new Color(190, 185,
230));
        UIManager.put("Button.foreground", new Color(30, 25,
60));

        setTitle("Рассчитать судьбу по дню недели");
        setSize(750, 500);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        loadBackgroundImage();

        JPanel backgroundPanel = new JPanel() {
            @Override
            protected void paintComponent(Graphics g) {
                super.paintComponent(g);
                if (backgroundImage != null) {
                    g.drawImage(backgroundImage, 0, 0,
getWidth(), getHeight(), this);
                }
            }
        };
        backgroundPanel.setLayout(new BorderLayout());

        backgroundPanel.setBorder(BorderFactory.createEmptyBorder(30,
40, 40, 40));
        setContentPane(backgroundPanel);

        JLabel title = new JLabel("РАССЧИТАТЬ СУДЬБУ ПО ДНЮ
НЕДЕЛИ");
        title.setHorizontalAlignment(SwingConstants.CENTER); //
добавить эту строку
        title.setFont(new Font("Arial", Font.BOLD, 26));
        title.setForeground(ACCENT_YELLOW);
        title.setBorder(BorderFactory.createEmptyBorder(0, 0,
25, 0));
        backgroundPanel.add(title, BorderLayout.NORTH);

```

```

RoundedPanel mainPanel = new RoundedPanel(30);
mainPanel.setBackground(PANEL_PURPLE);
mainPanel.setLayout(new BorderLayout(20, 0));
mainPanel.setBorder(BorderFactory.createEmptyBorder(25,
25, 25, 25));

JPanel leftPanel = new JPanel(new BorderLayout(0, 15));
leftPanel.setOpaque(false);

RoundedPanel buttonWrapper = new RoundedPanel(20);
buttonWrapper.setBackground(FIELD_LIGHT);
buttonWrapper.setLayout(new BorderLayout());

inputDateButton = new JButton("ВВЕСТИ ДАТУ РОЖДЕНИЯ");
inputDateButton.setFont(new Font("Arial", Font.BOLD,
16));
inputDateButton.setForeground(ACCENT_YELLOW);
inputDateButton.setBackground(FIELD_LIGHT);
inputDateButton.setBorderPainted(false);
inputDateButton.setFocusPainted(false);
inputDateButton.setContentAreaFilled(false);

inputDateButton.setBorder(BorderFactory.createEmptyBorder(15,
20, 15, 20));

inputDateButton.setCursor(Cursor.getPredefinedCursor(Cursor.HAND
_CURSOR));
buttonWrapper.add(inputDateButton, BorderLayout.CENTER);

leftPanel.add(buttonWrapper, BorderLayout.NORTH);

RoundedPanel resultWrapper = new RoundedPanel(20);
resultWrapper.setBackground(FIELD_LIGHT);
resultWrapper.setLayout(new BorderLayout());

resultArea = new JTextArea("Судьба не вычислена");
resultArea.setWrapStyleWord(true);
resultArea.setLineWrap(true);
resultArea.setEditable(false);
resultArea.setOpaque(false);
resultArea.setFont(new Font("Arial", Font.PLAIN, 14));

```

```

        resultArea.setForeground(TEXT_DARK);
        resultArea.setBorder(BorderFactory.createEmptyBorder(15,
15, 15, 15));

        JScrollPane scrollPane = new JScrollPane(resultArea);
        scrollPane.setBorder(BorderFactory.createEmptyBorder());
        scrollPane.setOpaque(false);
        scrollPane.getViewPort().setOpaque(false);
        resultWrapper.add(scrollPane, BorderLayout.CENTER);

        leftPanel.add(resultWrapper, BorderLayout.CENTER);

        mainPanel.add(leftPanel, BorderLayout.CENTER);

        JLabel catLabel = createCatLabel();
        if (catLabel != null) {
            mainPanel.add(catLabel, BorderLayout.EAST);
        }

        backgroundPanel.add(mainPanel, BorderLayout.CENTER);

        model.addChangeListener(evt -> {
            DateBirth updated = (DateBirth) evt.getNewValue();
            Fate fate =
FateRepository.findById(updated.getDayOfWeek());
            showResult(fate.getDescription());
        });
    }

    private void loadBackgroundImage() {
        URL url =
getClass().getClassLoader().getResource("resources/sky.jpg");
        if (url != null) {
            backgroundImage = new ImageIcon(url).getImage();
        }
    }

    private JLabel createCatLabel() {
        URL url =
getClass().getClassLoader().getResource("resources/cat.png");
        if (url == null) {

```

```

        return null;
    }
    ImageIcon icon = new ImageIcon(url);
    Image scaled = icon.getImage().getScaledInstance(180,
-1, Image.SCALE_SMOOTH);
    JLabel label = new JLabel(new ImageIcon(scaled));
    label.setBorder(BorderFactory.createEmptyBorder(0, 0, 0,
10));
    return label;
}

    public void
addInputDateListener(java.awt.event.ActionListener listener) {
        inputDateButton.addActionListener(listener);
    }

    public void showResult(String text) {
        resultArea.setText(text);
    }

    public void showError(String message) {
        JOptionPane.showMessageDialog(this, message, "Ошибка",
JOptionPane.ERROR_MESSAGE);
    }
}

```

DateInputDialog.java

```

package main.view;

import javax.swing.*.*;
import java.awt.*.*;

public class DateInputDialog extends JDialog {

    private static final Color BG_DARK = new Color(20, 25, 45);
    private static final Color ACCENT_YELLOW = new Color(255,
205, 60);
    private static final Color FIELD_LIGHT = new Color(190, 185,
230);
    private static final Color TEXT_DARK = new Color(30, 25,
60);

    private JTextField dayField;
    private JTextField monthField;
    private JTextField yearField;
    private JButton okButton;
}

```

```

private boolean confirmed = false;

public DateInputDialog(JFrame parent) {
    super(parent, "Ввод даты рождения", true);
    setSize(350, 260);
    setLocationRelativeTo(parent);

    JPanel panel = new JPanel(new GridLayout(4, 2, 10, 15));
    panel.setBackground(BG_DARK); // сплошной темный фон
    вместо картинки
    panel.setBorder(BorderFactory.createEmptyBorder(20, 20,
20, 20));
    setContentPane(panel);

    addStyledLabel(panel, "День:");
    dayField = createStyledField();
    panel.add(dayField);

    addStyledLabel(panel, "Месяц:");
    monthField = createStyledField();
    panel.add(monthField);

    addStyledLabel(panel, "Год:");
    yearField = createStyledField();
    panel.add(yearField);

    okButton = new JButton("OK");
    styleButton(okButton);

    panel.add(new JLabel());
    panel.add(okButton);

    okButton.addActionListener(e -> {
        confirmed = true;
        setVisible(false);
    });
}

private void addStyledLabel(JPanel panel, String text) {
    JLabel label = new JLabel(text);
    label.setForeground(ACCENT_YELLOW);
    label.setFont(new Font("Arial", Font.BOLD, 14));
    panel.add(label);
}

private JTextField createStyledField() {
    JTextField field = new JTextField();
    field.setBackground(FIELD_LIGHT);
    field.setForeground(TEXT_DARK);
    field.setFont(new Font("Arial", Font.PLAIN, 14));
    field.setBorder(BorderFactory.createEmptyBorder(5, 8, 5,
8));
    return field;
}

```

```

    }

    private void styleButton(JButton button) {
        button.setFont(new Font("Arial", Font.BOLD, 14));
        button.setForeground(ACCENT_YELLOW);
        button.setBackground(FIELD_LIGHT);
        button.setFocusPainted(false);
    }

    button.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR))
    ;

    }

    public void setValues(int day, int month, int year) {
        dayField.setText(String.valueOf(day));
        monthField.setText(String.valueOf(month));
        yearField.setText(String.valueOf(year));
    }

    public boolean isConfirmed() {
        return confirmed;
    }

    public void resetConfirmed() {
        confirmed = false;
    }

    public String getDayText() { return
dayField.getText().trim(); }
    public String getMonthText() { return
monthField.getText().trim(); }
    public String getYearText() { return
yearField.getText().trim(); }
}

```

FateRepository.java

```

package main.model;

import java.util.List;

public class FateRepository {

    private static final List<Fate> ALL = List.of(
        new Fate(0, "Люди, рожденные в понедельник, верные и
надежные, ценят уют в доме и могут быть очень романтичными. В
понедельник рождаются сильные люди, способные многого добиться.
Главное для людей понедельника понимать, чего они на самом деле
хотят. И если они найдут для себя цель, то обязательно ее
достигнут.\n" +
            "\n" +
            "Люди понедельника очень коммуникабельные,
могут найти общий язык с кем угодно. Они легко сходятся с
людьми, но несмотря на эту легкость иногда их сложно понять.

```

Настроение людей понедельника весьма переменчиво. Рожденные в понедельник очень артистичные натуры, обладают сильным творческим потенциалом. У них часто встречается талант художника.") ,

new Fate(1, "Во вторник рождаются воины, защитники, захватчики. Физическая сила играет в жизни этих людей не последнюю роль.\n" +

"\n" +

"Люди вторника отличаются мужеством и стойкостью перед невзгодами. Они выносливы и надежны. Ставя перед собой цель, они уже не отступают. Иногда они излишне максималисты, но зато, если они что-то начинают делать, то делают это очень хорошо, стремясь к идеалу. Они вообще не возьмутся за дело, если не будут уверены, что смогут сделать хорошо.\n" +

"\n" +

"Рожденные во вторник требовательны к себе и окружающим. Категоричны в суждениях и терпеть не могут возражений. Их импульсивность может стать причиной для неприятностей. Они могут совершать поступки, о которых потом сожалеют. С такими людьми сложно ужиться, потому у людей вторника часто не складывается личная жизнь. Зато их целеустремленность позволяет построить хорошую карьеру.") ,

new Fate(2, "Среда очень противоречивый день. Родившиеся в среду до обеда не очень удачливы. Им сложно добиться успеха и процветания. А люди, рожденные после полудня, наоборот, могут достичь многого.\n" +

"\n" +

"Среда наделяет людей умом и способностью выражать свои мысли. Люди среды коммуникабельны, отличаются хитростью и ловкостью. Они могут строить карьеру в абсолютно разных направлениях, особенно там, где требуется холодный расчет. Торговля, сфера услуг, собственный бизнес, фриланс и даже писательское дело – им доступно все.") ,

new Fate(3, "Четверг приносит удачу тем, кто родился в этот день. В четверг рождаются люди с богатым воображением. Они сумеют найти выход даже из самой запутанной ситуации.\n" +

"\n" +

"Они очень привлекательны для окружения, обладают сильной энергетикой и часто становятся лидерами. Люди четверга щедрые и добрые, часто увлекаются духовными практиками или посвящают себя эзотерике.") ,

new Fate(4, "В пятницу рождаются люди, любящие перемены. Люди пятницы очень любознательные, этикие ищущие натуры с сильной тягой к путешествиям и приключениям.\n" +

"\n" +

"Пятница наделяет людей, способностью любить и быть любимыми. Рожденные в пятницу высоко ценят семью, любовь. Для них настоящие чувства могут быть и даром судьбы, и роковым испытанием.\n" +

"\n" +

"Они высоко ценят уют и комфорт, всегда стремятся к благополучию. Но им сложно принадлежать одному

```

человеку. Их тяга к исследованиям и приключениям
распространяется и на природу человеческих отношений.\n" +
    "\n" +
    "Люди пятницы наделены множеством талантов,
но особенно хороши они в творческих профессиях. Артисты,
художники, писатели – их может ожидать успех во многих
направлениях.\n" +
    "\n" +
    "При выборе работы люди пятницы
руководствуются прежде всего выгодностью места в плане дохода. И
часто им удается найти удачное для себя место без особых усилий.
Им везет по жизни, в том числе и в финансах."),
    new Fate(5, "В субботу рождаются талантливые,
трудолюбивые и ответственные люди. Они умны и часто добиваются
высокого положения в карьере. Они исполнительны и аккуратны и
потому им хорошо даются точные науки.\n" +
    "\n" +
    "Но им никогда ничего не дается просто так.
Успеха они добиваются своим трудом, причем часто к успеху идут
долгой и сложной дорогой.\n" +
    "\n" +
    "И если в карьере они могут добиться высот,
то в личной жизни им часто не везет. Рожденные в субботу часто
одиночки по жизни."),
    new Fate(6, "В воскресенье рождаются люди с
сильнейшим творческим началом и очень развитой интуицией. У
людей, рожденных в воскресенье, невероятно сильная связь с
высшими силами.\n" +
    "\n" +
    "Люди воскресенья уверенные в себе и своем
будущем. Они оптимисты, с радостью готовы делиться своим
солнечным настроением с окружающими. В их голове всегда
рождается много идей. Конечно, не все идеи будут реализованы, но
сами люди воскресенья не сожалеют о том, что у них что-то не
получилось сделать, они смело идут вперед, ведь в их голове
рождаются все новые и новые планы.\n" +
    "\n" +
    "Рожденные в воскресенье очень харизматичны
и привлекательны для окружающих. Они чаще всего выбирают
творческие профессии и, если по воле случая профессия у них не
творческая, значит у них есть хобби, так или иначе связанное с
творчеством.")
);

public static Fate findById(int id) {
    if (id < 0 || id >= ALL.size()) {
        throw new IllegalArgumentException("Судьба с id=" +
id + " не найдена");
    }
    return ALL.get(id);
}
}

```

Fate.java

```
package main.model;

public class Fate {

    private int id;
    private String description;

    public Fate(int id, String description) {
        this.id = id;
        this.description = description;
    }

    public int getId() {
        return id;
    }

    public String getDescription() {
        return description;
    }
}
```

DateBirth.java

```
package main.model;

import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;
import java.time.DateTimeException;
import java.time.LocalDate;

public class DateBirth {

    public static final String DATE_CHANGED = "date";

    private final PropertyChangeSupport support = new
PropertyChangeSupport(this);

    private int day;
    private int month;
    private int year;

    public DateBirth() {
        this.day = 1;
        this.month = 1;
        this.year = 2000;
    }

    public DateBirth(int day, int month, int year) {
        this.day = day;
    }
}
```

```

        this.month = month;
        this.year = year;
    }

    public void addChangeListener(PropertyChangeListener
listener) {
        support.addPropertyChangeListener(DATE_CHANGED,
listener);
    }

    public void setDate(int day, int month, int year) {
        this.day = day;
        this.month = month;
        this.year = year;
        support.firePropertyChange(DATE_CHANGED, null, this);
    }

    public int getDay() {

        return day;
    }

    public int getMonth() {

        return month;
    }

    public int getYear() {

        return year;
    }

    public int getDayOfWeek() {
        LocalDate date = LocalDate.of(year, month, day);
        return date.getDayOfWeek().getValue() - 1;
    }

    public boolean isValid() {
        try {
            LocalDate date = LocalDate.of(year, month, day);
            return !date.isAfter(LocalDate.now());
        } catch (DateTimeException e) {
            return false;
        }
    }
}

```

MainController.java

```
package main.controller;
```

```

import main.model.DateBirth;
import main.view.DateInputDialog;
import main.view.MainForm;

public class MainController {

    private final DateBirth model;
    private final MainForm view;

    public MainController(DateBirth model, MainForm view) {
        this.model = model;
        this.view = view;

        view.addInputDateListener(e -> onInputDate());
    }

    private void onInputDate() {
        DateInputDialog dialog = new DateInputDialog(view);
        dialog.setValues(model.getDay(), model.getMonth(),
model.getYear());

        while (true) {
            dialog.setVisible(true);

            if (!dialog.isConfirmed()) {
                return;
            }

            try {
                int day = Integer.parseInt(dialog.getDayText());
                int month =
Integer.parseInt(dialog.getMonthText());
                int year =
Integer.parseInt(dialog.getYearText());

                DateBirth candidate = new DateBirth(day, month,
year);
                if (!candidate.isValid()) {
                    view.showError("Введена некорректная
дата.");
                    dialog.resetConfirmed();
                    continue;
                }

                model.setDate(day, month, year);
                return;

            } catch (NumberFormatException e) {
                view.showError("Все поля должны быть числами.");
                dialog.resetConfirmed();
            }
        }
    }
}

```